

OBJECT-ORIENTED PROGRAMMING (OOP)

Senior Software Engineer / Technical Lead Interview Prep – 2–3 Minute Review

1 WHY



Manage change by grouping state/behavior behind contracts.



Model collaborating responsibilities in a system.



Enable *substitution* and *modular ownership*.

2 CORE IDEA

Objects have *identity*, *state*, *behavior*.
Classes/Types define *contracts*.
Messages/Method calls coordinate.



Four core concepts (with caveats)



Encapsulation
Bundle data + methods;
hide representation.
Caveat: avoid over-hiding.



Abstraction
Expose essentials, hide irrelevant details.
Caveat: don't leak abstractions.



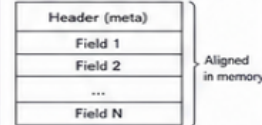
Polymorphism
Same message, different implementations.
Caveat: can hide cost/complexity.



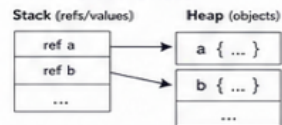
Inheritance
Share structure and behavior.
Caveat: tightens coupling; prefer composition.

3 INTERNALS

Object Layout (typical)



Stack vs Heap



Dispatch Mechanism



Allocation & Lifetime

- new / constructor or factory
- GC / ref counting / RAII / arenas
- Destructors / finalizers release resources

Construction & Invariants

- Constructors create valid state.
- Maintain invariants across all public methods.

Dispatch

Static (early binding): resolved at compile time.
Dynamic (late binding): resolved at runtime via vtable/itable.

Escape & Locality

- Local objects (don't escape): good locality, stack allocation, easier optimization.
- Escaped objects: heap allocation, pointer chasing, poorer locality.

4 COMPLEXITY

OOP does NOT change algorithmic Big O.
It can affect constant factors and memory.

Cost Source	Impact	Notes / Mitigation
Indirection	↑	Pointer chasing; hurts cache
Allocation	↑	Heap alloc/free overhead; GC pressure
Dispatch (virtual call)	↑	vtable lookup; may block inlining
Cache Misses	↑↑	Poor locality in object graphs
Synchronization	↑↑	Locks, contention

- Mitigations**
- Devirtualization, inlining, escape analysis
 - Object pooling / arenas, batching
 - Better data locality, fewer indirections

Value Objects / Data-Oriented Design

	Value Objects (VO)	Data-Oriented (SoA)
Identity	None (by value)	None
Mutability	Prefer immutable	Often immutable / bulk
Layout	Contiguous possible	Struct of arrays
Best For	Small, self-contained concepts	Large homogeneous data; hot loops, ECS
Trade-off	Copy cost if large	Less expressive behavior

Choose the simplest model that meets requirements and performance goals.

5 TRADE-OFFS

What OOP Gives You

- ✓ Modular contracts
- ✓ Testability via boundaries
- ✓ Extensibility via substitution
- ✓ Clear ownership of responsibilities

Risks / Costs

- ✗ Over-abstraction / premature generalization
- ✗ Large mutable object graphs
- ✗ Inheritance coupling and fragile base classes
- ✗ Allocation and poor locality

When Not to Use OOP as Primary Model

- Pure transformations / stateless functions
- Numeric hot loops / SIMD-friendly code
- Simple data pipelines / ETL
- ECS or data-oriented workloads
- When behavior is trivial and data dominates

OOP vs Functional / Data-Oriented

Aspect	OOP	Functional	Data-Oriented
State	Encapsulated, mutable	Immutable	External, structured
Change	Methods mutate or replace	Transform and return	Bulk operations
Composition	Objects & refs	Functions	Data + systems
Best For	Complex domains, responsibilities	Transformations, concurrency	Performance, scale, ECS

6 DISCUSSION

Q: Class vs Object?	Class = blueprint/type; Object = runtime instance.
Q: What is encapsulation?	Bundling state and behavior; controlling access.
Q: Abstraction vs Information Hiding?	Abstraction = what is shown. Hiding = what is hidden.
Q: Interface vs Abstract Class?	Interface = contract only; Abstract class = partial implementation + state.
Q: Is polymorphism expensive?	Slight cost (indirection); usually negligible. Optimize hot paths (devirtualize / inline).
Q: Equality vs Identity?	Identity (who) = reference/id; Equality (what) = value equivalence (override equals/hashCode).
Q: Immutability?	Simplifies reasoning, thread-safety, caching. Favor for value objects.
Q: Anemic Domain Model?	Data + getters/setters with no behavior. Prefer rich model with behavior.
Q: Tell, Don't Ask?	Encapsulate behavior: ask object to do the work, don't expose internals to decide.

7 RELATED

★ SOLID Principles	SRP, OCP, LSP, ISP, DIP
⚙️ Composition Over Inheritance	Prefer "has-a" over "is-a"
🔗 Design Patterns	Strategy, Factory, Observer, Decorator, Adapter, etc.
📖 Domain-Driven Design (DDD)	Ubiquitous language, Aggregates, Entities, Value Objects, Repositories
🔌 Dependency Injection (DI)	Inversion of control, testability, loose coupling
λ Functional Programming	Pure functions, immutability, composition
🔲 Data-Oriented Design (DOD)	SoA, ECS, cache-friendly layouts
[] ADTs	Abstract Data Types, contracts independent of implementation

8 PRODUCTION USE CASES

🏢 Service Domain Models	Capture business rules, invariants, rich behavior.
🌐 Web/API Controllers	Handle requests, validate, orchestrate use cases.
📁 UI Components	Encapsulate state, events, rendering, composition.
🗄️ ORM Entities (Caution)	Be wary of lazy loading, N+1, anemic models, persistence leakage.
🔧 Plugins / Drivers	Interface contracts, dynamic loading, version tolerance.
💳 Payment Strategies	Strategy pattern for providers, substitution, retries, idempotency.
🔄 Workflow / State Machines	States with behavior, transitions, invariants.
🎮 Game Entities	Components, behaviors, interactions (ECS hybrid common).
☁️ SDK / API Clients	Encapsulate protocol, retries, rate limiting, auth.

9 INTERVIEW THINKING

- 1 Identify responsibilities and invariants.
- 2 Find natural boundaries and contracts.
- 3 Decide ownership and mutability.
- 4 Prefer composition; avoid unnecessary hierarchy.
- 5 Ensure substitutability (LSP) across boundaries.
- 6 Define failure and concurrency semantics.
- 7 Design test seams and dependency boundaries.
- 8 Consider runtime cost: allocation, dispatch, locality.
- 9 Avoid speculative generality; YAGNI.

10 SENIOR NOTES

- OOP is about responsibility design, not class multiplication.
- Prefer immutable value objects; keep them small.
- Encapsulate invariants, not just fields.
- Composition usually reduces coupling.
- Domain boundaries often differ from runtime/module boundaries—manage both.
- Profile allocation, dispatch, and locality in production.

11 MENTAL HOOK

"Good OOP hides decisions likely to change behind responsibilities that remain stable."

SOLID PRINCIPLES

Senior Software Engineer / Technical Lead Interview Prep — 2-3 Minute Review

1 WHY



Reduce change amplification and coupling.



Create substitutable, testable boundaries.



Provide a shared vocabulary for design review, not rigid laws.

Outcome

- ✓ Smaller change radius
- ✓ Defect isolation
- ✓ Testability and fast feedback
- ✓ Safer refactoring and replacement

4 COMPLEXITY

SOLID does NOT change Big O.

Abstraction introduces real costs:

- More types, indirection, allocations
- Build graph complexity
- Cognitive load

Benefits are measured by:



Change radius



Defect isolation



Testability



Evolvability



7 RELATED



OOP Fundamentals — classes, inheritance, polymorphism



Compositions over inheritance



Dependency Injection & Inversion of Control



Hexagonal (Ports & Adapters) Architecture



Cohesion / Coupling Principles



DDD — Bounded Contexts, Ubiquitous Language



Design Patterns — Strategy, Adapter, Factory, Decorator



YAGNI — You Aren't Gonna Need It

2 CORE IDEA



SRP
(Single Responsibility)

A class/module has one reason to change.



OCP
(Open/Closed)

Extend behavior without modifying the stable core.



LSP
(Liskov Substitution)

Subtypes must be substitutable; preserve invariants and contracts.



ISP
(Interface Segregation)

Clients should not be forced to depend on interfaces they don't use.



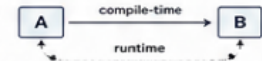
DIP
(Dependency Inversion)

High-level policy does not depend on details. Both depend on abstractions.

3 INTERNALS



Dependencies



Compile-time dependencies vs. runtime object graph.

Interfaces / Ports



Define contracts at boundaries.

Factories / DI



Composition root wires concrete implementations.

Contracts

Preconditions: what must be true before call
Postconditions: what is guaranteed after call
Invariants: always true within boundary

Cohesion / Coupling

High cohesion within a module. Low coupling between modules.

Package / Module Boundaries

Explicit boundaries, stable dependencies inward, volatile dependencies outward.

4 COMPLEXITY

SOLID does NOT change Big O.

Abstraction introduces real costs:

- More types, indirection, allocations
- Build graph complexity
- Cognitive load

Benefits are measured by:



Change radius



Defect isolation



Testability



Evolvability



5 TRADE-OFFS

Optimize For

- ✓ Evolvability
- ✓ Replaceability
- ✓ Testability
- ✓ Parallel development

Risks / Costs

- ✗ Interface explosion
- ✗ Indirection & boilerplate
- ✗ Speculative abstractions
- ✗ Distributed ownership and coordination

When NOT to apply strictly

- ✗ Stable, tiny code with no expected change
- ✗ Simple data transforms, scripts, one-offs
- ✗ Hot loops / performance critical inner code
- ✗ Premature abstractions or frameworks

Use judgment. YAGNI.

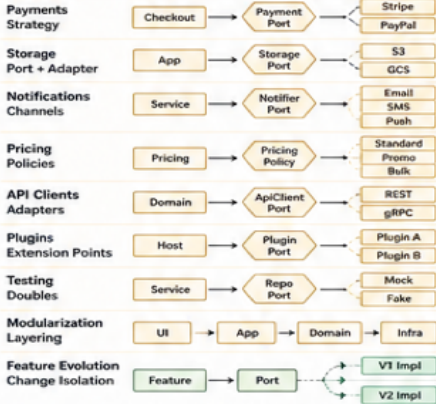
6 DISCUSSION

- 2 SRP: whose reason to change?
- 2 Is OCP realistic or over-engineered?
- 2 LSP: rectangle-square problem?
- 2 ISP: how fine should interfaces be?
- 2 DIP vs. DI — what's the difference?
- 2 Who owns abstractions — providers or consumers?
- 6 Do mocks create hidden coupling?
- 7 When is it right to violate a principle consciously?

Common Code Smells

```
class GodService {  
    // does everything  
}  
  
if (type == 'A') { ... }  
if (type == 'B') { ... }  
  
class Square : Rectangle {  
    setWidth(w) { setHeight(w); // breaks LSP  
}  
  
interface FatInterface {  
    a(); b(); c(); /* d(); e(); */  
}  
  
class OrderService {  
    private new SqlRepo(); // detail leakage  
}  
  
Who decides the interface?  
Consumer needs drive it.  
  
Test asserts internal calls  
instead of observable behavior.  
  
Document rationale, measure impact,  
and keep it local and visible.
```

8 PRODUCTION EXAMPLES



9 INTERVIEW THINKING

- 1 Identify change axes
What is likely to change? Why? How often?
- 2 Define responsibility boundaries
Who cares about what? Single reason to change.
- 3 Define contracts (Substitution)
What must always be true? Pre/Post/Invariants.
- 4 Create client-specific interfaces (ISP)
Only what clients need, no more.
- 5 Set dependency direction (DIP)
High-level depends on abstractions.
- 6 Compose in composition root
Wire implementations via DI/Factories.
- 7 Evaluate runtime cost
Indirection, allocs, caches, contention, latency.
- 8 Gather evidence / test
Prove behavior, not implementation.
- 9 Choose simplest viable abstraction
Start small, evolve with real change.

10 SENIOR NOTES



Principles optimize for change, not for the number of classes.



Abstractions should belong to consumers, not providers.



LSP is about behavioral substitutability, not type shape.



SRP depends on organizational change forces and team topology.



Mocks can fossilize implementation and create brittle tests.



Violations are acceptable when deliberate, local, and measured. Document the trade-off and monitor it.

Quick Check

Question	Good Signal
Will this change?	If yes, isolate it.
Who needs this?	Design for them.
Can I replace it?	If yes, depend on an abstraction.
Can I test it in isolation?	If not, tighten boundaries.
What's the simplest thing that works?	Prefer less abstraction.

11 MENTAL HOOK

SOLID keeps policy stable by pushing volatile details behind honest contracts.



- S** Single Responsibility
- O** Open/Closed
- L** Liskov Substitution

- I** Interface Segregation
- D** Dependency Inversion

COMPOSITION vs INHERITANCE

Engineering Map for Senior Software Engineer / Technical Lead Interview Prep

1 WHY

Choose the reuse/variation mechanism that controls coupling and preserves substitutability.

- The goal is not code reuse, it's the right abstraction boundary.
- Inheritance couples at design time across the entire hierarchy.
- Composition couples at assembly time and can be reconfigured.
- Preserve substitutability (LSP) while minimizing ripple effects of change.
- Prefer the simplest model that meets today's and tomorrow's constraints.



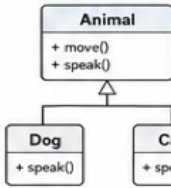
Default to composition.

Use inheritance when there is a true "is-a" relationship with a stable contract.

2 CORE IDEA

INHERITANCE (is-a)

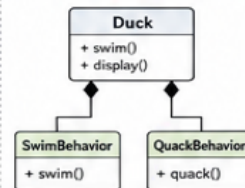
Subtype with substitutable contract.



Dog is-a Animal
Can substitute anywhere Animal is expected.

COMPOSITION (has-a / uses-a)

Delegates behavior to composed objects.



Duck has-a behaviors
Behaviors are swappable at runtime.

is-a → subtype, inheritance, substitutability
has-a / uses-a → composition, delegation, flexibility

3 INTERNALS



Inheritance dispatch

- Virtual call → vtable lookup (or devirtualized)
- Override provides polymorphic behavior



Base-class layout & contracts

- Defines public/protected API & invariants
- Changing base breaks all derived clients



Construction & protected state

- Base ctor runs before derived ctor
- Protected state couples hierarchy



Composition object graph

- Objects collaborate via references
- Delegation forwards calls to collaborators



Interfaces as boundaries

- Depend on abstractions, not concretions
- Enables substitutable and swappable parts



DI / Factories / Service Locators

- Control assembly, not implementation
- Choose implementations at runtime



Ownership & lifetime

- Who creates, owns, and destroys?
- Clear lifetimes prevent leaks & cycles

4 COMPLEXITY

Aspect	Inheritance	Composition
Asymptotic Cost	Same Big O generally	Same Big O generally
Dispatch	Virtual call (vtable) may block inlining	Extra indirection (call through field)
Dependencies	Rigid, compile-time & transitive	Wired at runtime (softer coupling)
Object Allocation	One object	More objects (components)
Memory Layout	Fixed by hierarchy	Flexible graph
Inlining / Devirt	Possible with final/sealed & whole prog analysis	Possible through interfaces & JIT heuristics

Complexity Beyond Big O

Dimension	Inheritance	Composition
Cognitive	High when hierarchies grow	Moderate, local reasoning
Change Impact	Wide (ripple down tree)	Local (replace component)
Testability	Harder (needs subclassing)	Easier (mock/replace dep)
Extensibility	Closed by base design	Open via new components

Rule of thumb: Same runtime cost class. Different human cost.

5 TRADE-OFFS

INHERITANCE — Strengths

- ✓ Concise; share code & contracts
- ✓ Models stable taxonomies (domain "kinds")
- ✓ Enables framework hooks & polymorphism

INHERITANCE — Risks

- ✗ Fragile Base Class problem
- ✗ Tight coupling, ripple changes
- ✗ Combinatorial explosion of subclasses
- ✗ Deep hierarchies are hard to understand

COMPOSITION — Strengths

- ✓ Runtime flexibility (swap/reconfigure)
- ✓ Test seams & isolation
- ✓ Localized change, smaller blast radius
- ✓ Mix behaviors orthogonally

COMPOSITION — Costs

- ✗ More wiring, indirection & objects
- ✗ Boilerplate & delegation code
- ✗ Potential performance overhead
- ✗ Design effort to define boundaries

DECISION GUIDE

Question	IF YES →	IF NO →
Is there a true is-a relationship?	Consider inheritance	Prefer composition
Must callers treat them interchangeably?	Inheritance + LSP	Composition
Is the hierarchy closed & stable?	Inheritance OK	Composition
Do we need to mix behaviors?	Composition	Inheritance
Will implementations change often?	Composition	Inheritance
Need to test in isolation?	Composition	Inheritance

6 DISCUSSION

- **LSP (Substitutability)?** Can every subclass be used wherever the base is expected without changing correctness of the program?
- **When is inheritance valid?** When the subtype relation is real, the contract is stable, and behavior varies primarily by overriding.
- **Template Method pattern?** Inheritance to define algorithm skeleton; subclasses override specific steps.
- **Multiple inheritance / mixins / traits?** Increases complexity and coupling; prefer mixins/traits or composition for capabilities.
- **Delegation vs inheritance?** Prefer delegation (composition) unless subtype polymorphism is required by clients.
- **Strategy pattern?** Use composition to select algorithms at runtime through a strategy interface.
- **Base class versioning?** Hard to evolve without breaking subclasses. Favor sealed/closed hierarchies or explicit versioned contracts.
- **Protected members?** Coupling leak. Forces knowledge of internal details into subclasses.
- **Sealed/final classes?** Constrain extension to control invariants and enable optimization.
- **Open/Closed Principle?** Inheritance often hard to keep open. Composition with abstractions is easier to extend.

7 RELATED

	OOP Basics	Classes, objects, polymorphism, abstraction, encapsulation.
	SOLID / LSP	LSP, OCP, ISP, DIP guide both inheritance and composition.
	Strategy	Encapsulate algorithms; select at runtime via composition.
	Decorator	Add responsibilities dynamically without subclass explosion.
	Template Method	Define skeleton in base class; subclasses fill the steps.
	DI (Dependency Injection)	Inversion of control for wiring and testability.
	Traits / Mixins	Reuse behavior without deep inheritance chains.
	Delegation	Forwarding calls to another object that does the work.
	ADTs / Interfaces	Define what; hide how. Promote substitutability.

8 PRODUCTION EXAMPLES

	Payment strategies	Different gateways (Stripe, PayPal, Adyen) via Strategy.
	Storage adapters	S3, GCS, Azure, local FS behind Storage interface.
	UI frameworks	Base classes for framework hooks (e.g. View, Controller) careful: inheritance locks you in.
	Middleware decorators	Add logging, auth, caching, retries via decorators.
	Game entity components	Entity composed of components (Physics, Render, AI).
	Notification channels	Email, SMS, Push, Slack via channel strategy composition.
	Parsers / serializers	JSON, XML, Protobuf via parser strategies.
	Policy / rules engines	Compose validation, pricing, authorization policies.
	Plugin architectures	Host defines interfaces; plugins loaded & composed.

9 INTERVIEW THINKING

- 1 Identify the domain model. What are the core concepts?
- 2 Is there a true subtype contract? (is-a, substitutable?)
- 3 Perform substitutability tests (LSP lite):
 - Same observable behavior?
 - No stronger preconditions?
 - No weaker postconditions?
 - Invariants preserved?
- 4 What are the variation axes? Algorithms? Policies? Capabilities?
- 5 What owns what? Define lifetimes and boundaries.
- 6 Do we need runtime swapping or configuration? Testability?
- 7 How deep will the hierarchy be? Can we keep it shallow?
- 8 If composition, how will we wire dependencies? (DI/factory)
- 9 Any performance constraints or hot paths? Profile assumptions.

After analysis → choose the simplest design that satisfies requirements with minimal coupling and maximal flexibility.

10 SENIOR NOTES

- ★ **Reuse alone is not reason for inheritance.** Behavior + contract + substitutability are.
- ★ **Inherit contracts, compose capabilities.** Model what something is, not what it can do.
- ★ **Base classes are published APIs.** Treat them with semantic versioning discipline.
- ★ **Shallow, well-designed, sealed hierarchies** can be excellent. Depth is the problem.
- ★ **Composition does not eliminate coupling, it relocates it.** Design boundaries carefully.
- ★ **Prefer telling, don't asking.** Favor data-driven polymorphism when variants are closed and known (e.g., state machines, lookup tables).
- ★ **Optimize for change.** Assume requirements will evolve.
- ★ **Favor clarity over cleverness.** Future readers > future you.
- ★ **Measure when in doubt.** Don't prematurely micro-optimize.
- ★ **Document intent.** Future maintainers need the "why", not just the "how".

11 MENTAL HOOK

"Inheritance commits to identity; composition assembles capability."

DESIGN PATTERNS

Engineering Map for Senior Software Engineer / Technical Lead Interview Prep (2–3 minute review)

1 WHY



Design patterns give us **shared vocabulary** for recurring collaboration and change problems.



They capture **proven trade-offs** so teams can move faster and make better decisions.



Not copy-paste recipes. It's about understanding the problem, context, and costs.

Goal: communicate intent, align teams, and manage change cost over time.

2 CORE IDEA

A pattern describes a solution schema:

Context + **Forces** + **Structure** + **Consequences**

Categories



Creational
How objects are created

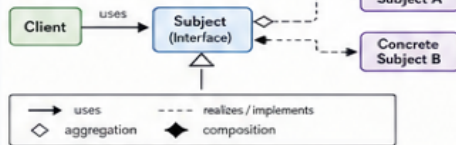


Structural
How objects are composed



Behavioral
How objects communicate

Tiny collaboration diagram (example)



3 INTERNALS



Object roles
Who participates and what each role is responsible for.



Dependency direction
Who knows/creates/calls whom.



Lifecycle
Creation, ownership, scope, and destruction.



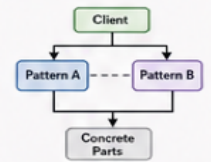
Delegation
Who does the real work.



Indirection
Add a level of separation to vary behavior or structure.

Pattern composition

Patterns combine. Example: "Command + Memento" for undo/redo; "Factory + Strategy" for pluggable algorithms.



Language / framework features can make patterns native—or unnecessary.

4 COMPLEXITY

Patterns rarely change asymptotic complexity (Big O), but they do add overhead in multiple dimensions.

Pattern	Objects (+)	Calls (+)	Alloc. (+)	State Trans. (+)	Cognitive / Build (+)	Runtime (+)
Factory	•	•	•	○	•	○
Builder	••	•	••	•	•	○
Adapter	•	•	○	○	•	○
Decorator	••	••	•	○	••	•
Facade	•	•	○	○	•	○
Strategy	•	•	○	○	•	○
Observer	••	••	•	••	••	•
Command	••	•	•	•	••	•
State	••	•	•	••	••	•
Template Method	•	•	○	○	•	○

○ Low • Moderate •• High ••• Very High

5 TRADE-OFFS

What you gain

- ✓ Shared vocabulary & alignment
- ✓ Extensibility / variation points
- ✓ Testability & seams
- ✓ Isolation of change
- ✓ Reusability of solutions

What you pay

- ✗ More objects, indirection, and wiring
- ✗ Harder to read for simple cases
- ✗ Overengineering & yak-shaving
- ✗ Potential runtime overhead

When NOT to use a pattern

- Direct code is clear and obvious
- Only one implementation/variant
- Behavior is stable and unlikely to change
- Language or framework feature already solves it
- Team unfamiliarity outweighs the benefits

6 DISCUSSION

- ❓ **Strategy vs State?** Strategy is external choice; State is internal change of an object's behavior and transitions.
- ❓ **Adapter vs Facade?** Adapter converts an interface for one client; Facade offers a simplified interface to a subsystem.
- ❓ **Decorator vs inheritance/proxy?** Decorator adds behavior dynamically; inheritance is static and rigid; proxy controls access / adds cross-cutting concerns.
- ❓ **Factory vs DI?** Factory hides creation behind an interface; DI (IoC) provides objects and manages wiring/lifecycle.
- ❓ **Observer failure modes?** Leaks, storms, slow/failed subscribers, ordering, reentrancy, eventual consistency.
- ❓ **Singleton problems?** Global state, hidden deps, hard to test, concurrency issues, lifecycle/teardown complexity.
- ❓ **Command benefits?** Decouples invoker/receiver; queues, logging, undo/redo, retries, scheduling, idempotency.
- ❓ **How do you choose?** Start from forces & trade-offs, consider alternatives, pick simplest that meets needs.

7 RELATED



SOLID Principles
SRP, OCP, LSP, ISP, DIP guide good design.



Composition over inheritance
Prefer objects + delegation.



Dependency Injection (DI)
Inversion of Control, containers.



Hexagonal / Ports & Adapters
Isolate domain from infrastructure.



Domain-Driven Design (DDD)
Bounded contexts, aggregates.



Refactoring
Simplify, extract, introduce patterns.



Functional Patterns
Immutability, pure functions, FP combinators.



GoF (Gang of Four)
Classic catalog of 23 patterns.

8 PRODUCTION



Provider strategies PaymentProvider (Stripe, PayPal, Adyen)



API adapters REST, SOAP, gRPC, legacy SDKs



Middleware decorators Logging, metrics, auth, caching



UI observers UI updates on model/data changes



Workflow states Order: Pending → Paid → Shipped → Done



Job commands Enqueue, retry, cancel, audit



Client facades SDK that hides multiple services



Configuration builders Complex objects with many options



Plugin factories Load plugins/modules/providers



Circuit breaker (resilience) Protect downstream dependencies

9 INTERVIEW THINKING

- 1 Name the change pressure / problem.
- 2 Identify forces (constraints, quality attributes).
- 3 Sketch the simplest direct design (no pattern yet).
- 4 Identify roles and responsibilities.
- 5 Determine dependencies and lifecycle.
- 6 Consider failure modes & concurrency.
- 7 Evaluate costs vs benefits.
- 8 Choose or compose the right pattern(s).
- 9 Explain alternatives and why you chose this one.



Think in trade-offs, not in pattern names.

10 SENIOR NOTES

- ★ Start from the problem, not the pattern name.
- ★ Patterns encode trade-offs—know your costs.
- ★ Distributed Observer needs delivery semantics (at-least-once, ordering, retries, idempotency).
- ★ Singleton is about global lifecycle and state—be suspicious; prefer scoped/managed lifetimes.
- ★ Framework annotations can hide patterns (e.g., DI, proxies, interceptors). Understand what's generated.
- ★ Refactor toward a pattern after repeated change, not before. YAGNI still applies.

Pattern smells

Overuse	Every problem becomes a pattern.
Misapplied	Forces don't match the context.
Leaky abstraction	Details leak across boundaries.
Premature	Added complexity with no benefit.
Magic indirection	Hard to debug, hard to reason.

11 MENTAL HOOK

“A design pattern is a named trade-off for a recurring shape of change.”

Quick legend

→ uses / calls

--- realizes / implements

◊ aggregation (shared)

◆ composition (owned)

DEPENDENCY INJECTION

ENGINEERING MAP FOR SENIOR SOFTWARE ENGINEER / TECHNICAL LEAD INTERVIEW PREP

1 WHY

- Separate object use from construction.
- Invert control of creating dependencies.
- Replace implementations & configuration without changing consumers.
- Enable test seams with fakes/mocks.
- Manage object lifecycles and scopes consistently.

WITHOUT DI

```
class OrderService {
  private PaymentGateway gw =
    new StripeGateway();
  void submit(Order o) { gw.charge(o); }
}
```

WITH DI (CONSTRUCTOR)

```
class OrderService {
  private final PaymentGateway gw;
  OrderService(PaymentGateway gw) {
    this.gw = gw;
  }
  void submit(Order o) { gw.charge(o); }
}
```

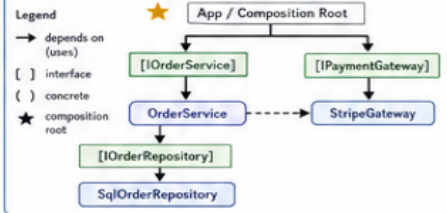


Outcome: Decoupled, testable, replaceable, and lifecycle-aware software with explicit boundaries.

2 CORE IDEA

- Dependencies are supplied to a consumer from the outside.
- Common injection forms (in order of preference):
 - Constructor Injection (preferred) — required deps.
 - Method Injection — operation-specific deps.
 - Property/Setter Injection — optional or reconfigurable deps.
- Composition Root wires the concrete object graph.
- DI is a technique.
- Container (DI framework) is a tool to implement it.
- Dependency Inversion Principle (DIP) is the principle.

OBJECT GRAPH (EXAMPLE)



4 COMPLEXITY

- Same algorithmic complexity (Big O) as without DI.
- Added overhead sources:
 - Startup: graph build, reflection or codegen.
 - Allocations: container structures, call sites, proxies.
 - Resolution: lookup cost (often O(1) average), interceptor overhead if any.
 - Memory: kept instances per scope + container metadata.
 - Large graphs: more startup time and memory footprint.
- Constructor injection uses direct dispatch—no runtime lookup on each call (after construction).

COST SNAPSHOT (RELATIVE)

Cost Area	Low	Medium	High	Notes
Startup Time		✓	✓	Reflection > codegen > manual
Resolve Time	✓	✓		Cached: ~O(1)
Per Call Overhead	✓			Constructor inj: none
Memory Footprint	✓	✓	✓	Depends on scopes/graph
Indirection Risk		✓	✓	Proxies, decorators, lazy

Tip: Prefer constructor injection and keep graphs shallow.

5 TRADE-OFFS

WHAT YOU GAIN

- Replaceability (swap implementations, environments).
- Testability with fakes/mocks.
- Centralized lifecycle & configuration.
- Explicit dependency contracts.
- Separation of concerns and modularity.

WHAT IT COSTS

- Indirection and abstraction layers.
- "Magic" wiring can be hard to trace.
- Startup failures if misconfigured.
- Scope leaks and lifetime mismatches.
- Interface/proliferation risk.

WHEN NOT TO USE (OR USE SPARINGLY)

- Simple value objects / pure functions.
- Trivial local dependencies (e.g., new helper()).
- Performance-critical hot paths using container lookups.
- Over-engineering small scripts / throwaway tools.
- When manual construction is clearer and cheaper.

3 INTERNALS

- Registration: map abstraction → implementation/lifetime.
- Resolution: build object graph, satisfy dependencies.
- Constructor discovery via reflection or compile-time codegen.
- Scopes/Lifetimes control instance reuse & disposal.

COMMON LIFETIMES

Scope	Description	Typical Use
Singleton	One instance per container (or app)	Stateless services, caches, config, loggers
Scoped/Request	One per logical scope (e.g., web request)	DbContext, user context, per-request services
Transient	New instance each resolve	Lightweight, stateless, value-like services

- Cycle detection prevents infinite graphs.
- Disposal: container tracks & disposes (Disposable instances).
- Factories/Providers for runtime parameters and late binding.
- Qualifiers/Named/Keyed bindings for multiple implementations.
- Startup validation: build & validate graph early (fail fast).

ANTI-PATTERN: SERVICE LOCATOR

Avoid pulling from container inside business code. It hides dependencies and harms testability.

6 DISCUSSION (BE READY TO REASON)

Question	Good Answer Direction
DI vs DIP vs IoC?	DIP is the principle, DI is the technique. IoC is the broader idea of ceding control (includes DI, eventing, callbacks).
Constructor vs Property Injection?	Constructor = required deps, immutability, easier testing. Property for optional or reconfigurable deps (rare).
Container vs Manual DI?	Container saves wiring for complex apps. Manual (composition root) is simple, fast, and explicit—often enough.
Service Locator?	Avoid in domain code. Hides deps, makes testing & refactoring harder.
Scoped dependency into Singleton?	Avoid. Causes scope leaks or disposed objects. Use factories or redesign.
Circular Dependencies?	Smell. Break with interface segregation, mediator, events, or lazy/factories.
Factories & Runtime Parameters?	Use factories/providers for values known at runtime (e.g., userId, tenantId).
Testing without Mocks?	Use in-memory fakes, test doubles, contract tests, or real implementations with isolated infrastructure.
Compile-time DI?	Yes (e.g., Dagger, Hilt, .NET Source Gen, Kotlin Koin compile mode) for safety & speed.

7 RELATED CONCEPTS

	SOLID / DIP	Depend on abstractions, not concretes.
	Composition Root	Single place that composes the object graph.
	Factories / Abstract Factory	Create objects with runtime parameters.
	Ports & Adapters (Hexagonal)	Domain depends on ports; adapters implement them.
	Service Locator (Anti-pattern)	Global access to container; hides dependencies.
	Configuration	Externalize settings; inject strongly typed options.
	Lifecycle / Scopes	Manage creation, reuse, and disposal.
	Test Doubles	Mocks, stubs, fakes, spies for testing behavior.

8 IN PRODUCTION (EXAMPLES)

Area	Typical Injections / Benefits
Controllers / APIs	Services, validators, mappers, loggers, options
Application Services	Repositories, domain services, event bus
Repositories / DAOs	DbContext, connection factories, caches
Storage / External Systems	S3/Blob clients, queues, search clients
Payments / Billing	PaymentGateway, fraud checker, retry policy
HTTP Clients	HttpClientFactory/proxies, resilience policies
Messaging Handlers	Bus, serializers, consumers, DLQ publishers
Background Jobs / Workers	Job services, schedulers, metrics, configs
Per-request Contexts	UserContext, correlation id, localization
Plugin / Extension Systems	Plugin discovery, registries, capability services
Feature Flags / Config	IFeatureManager, IOptions<T>, flag providers
Observability	Loggers, tracers, metrics, health checks

COMPOSITION ROOT EXAMPLE

MANUAL (PSEUDOCODE)	CONTAINER REGISTRATION (EXAMPLE)
<pre>var repo = new SqlOrderRepository(connStr); var gateway = new StripeGateway(apiKey); var svc = new OrderService(repo, gateway, new RetryPolicy()); var app = new WebApp(svc, new Logger()); app.Run();</pre>	<pre>services.AddSingleton<StripeGateway>(); services.AddScoped<IOrderRepository, SqlOrderRepository>(); services.AddScoped<IOrderService, OrderService>(); services.AddHttpClient(); var app = BuildApp(services); app.Run();</pre>

DI FLOW



9 INTERVIEW THINKING CHECKLIST

- Identify the dependency and its lifetime characteristics.
- Define the consumer contract (interface/port) and responsibilities.
- Constructor-inject required dependencies.
- Choose the composition root and registration strategy.
- Define scopes and disposal boundaries.
- Handle runtime parameters via factories/providers.
- Validate the graph at startup (fail fast).
- Test behavior with fakes in-memory or contract tests.
- Measure cost (startup, memory) and keep it simple.

Always optimize for clarity and correctness first; optimize performance second.

10 SENIOR NOTES (WHAT GOOD LOOKS LIKE)

- Inject collaborators, not ambient data (no "service locator in disguise").
- Treat scopes as correctness boundaries, not just performance hints.
- Keep domain/application code container-agnostic.
- Prefer small, focused interfaces with real substitution or change reason (YAGNI applies to abstractions too).
- Validate the object graph at startup; fail early, fail loudly.
- Prefer constructor injection and immutability for thread safety.
- Avoid creeping dependencies; review the graph regularly.
- Manual DI (composition root) is often the right level of abstraction.
- Think in terms of boundaries, lifetimes, and contracts.
- Measure in production; don't guess the impact.

Simple, explicit, testable, observable, and evolvable.

11 MENTAL HOOK

“Dependency injection makes construction a boundary so behavior can stay focused.”

DOMAIN-DRIVEN DESIGN (DDD)

Senior Software Engineer / Technical Lead Interview Prep (2-3 Minute Review)

1 WHY



Align software model and language with a complex business domain.



Isolate differing meanings in different parts of the organization.



Manage change by localizing impact behind well-defined boundaries.

2 CORE IDEA

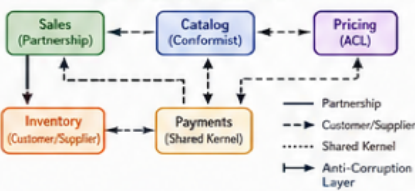
STRATEGIC (big picture)

- Domain & Subdomains (Core / Supporting / Generic)
- Ubiquitous Language (shared and precise)
- Bounded Contexts (semantic boundaries)
- Context Map (relationships between contexts)

TACTICAL (inside a context)

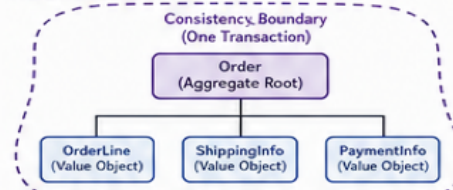
- Entities – identity, lifecycle, mutable
- Value Objects – immutable, value equality
- Aggregates – consistency boundary
- Domain Services – stateless domain logic
- Repositories – collection-like access
- Domain Events – facts that happened
- Factories – complex creation, invariants

EXAMPLE CONTEXT MAP



3 INTERNALS

AGGREGATE 101

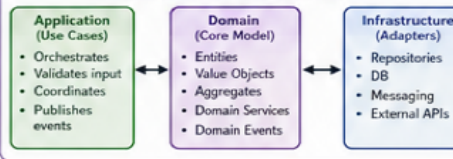


- Aggregate Root is the only entry point.
- Invariants enforced inside the boundary.
- Transactions never cross aggregate boundaries.

KEY INTERNALS

- Identity vs Value Equality**
Entities have identity; value objects compare by value.
- Repositories**
Mimic collection behavior; hide persistence details.
- Factories**
Encapsulate complex creation; ensure validity.
- Domain Events**
Inside domain to react, decouple; outside for integration.
- Anti-Corruption Layer (ACL)**
Translates between models; protects the domain.
- Layered Separation**
Application (use cases) → Domain → Infrastructure (Adapters, DB, Messaging, External APIs)

LAYERED ARCHITECTURE (EXAMPLE)



4 COMPLEXITY

DDD DOES NOT IMPROVE BIG O

- Adds modeling, types, mappings, and eventual consistency overhead.
- Reduces semantic complexity and change complexity.

SOURCES OF COMPLEXITY

Modeling Overhead	More types, abstractions, validation, and documentation.
Mapping Overhead	DTO ↔ Domain, persistence ↔ Domain, anti-corruption.
Eventual Consistency	Across aggregates and contexts; requires retries, idempotency.
Operational Complexity	More services/components, observability, deployments.

WHAT REDUCES COMPLEXITY

- ✓ Clear ubiquitous language reduces miscommunication.
- ✓ Bounded contexts localize change and meaning.
- ✓ Aggregates protect invariants and simplify reasoning.

REMEMBER

Aggregate size affects contention and load. Distributed boundaries add latency and failure modes.

5 TRADE-OFFS

BENEFITS	COSTS
Shared language	Upfront modeling cost
Team autonomy	More ceremony / discipline
High cohesion	Duplicated concepts across contexts
Isolated change	Eventual consistency challenges
Clear boundaries	More infrastructure / plumbing

WHEN NOT TO USE DDD

- CRUD apps or simple domains.
- Domain is unclear or volatile; no clear ownership.
- Small, short-lived prototypes or MVPs.
- Technical/algorithmic utility libraries.

6 DISCUSSION (COMMON INTERVIEW QUESTIONS)

- Q: Bounded context vs microservice?**
A: Bounded context is a semantic boundary; microservice is deployment choice. Many contexts can live in one service.
- Q: How do you choose aggregate boundaries?**
A: Around consistency/invariants and use cases. Keep them small; enforce invariants in a single transaction.
- Q: Entity vs value object?**
A: Entity has identity and lifecycle; value object is immutable, no identity, compared by value.
- Q: What is a domain service?**
A: Encapsulates domain logic that doesn't naturally belong to an entity or value object.
- Q: What is a repository?**
A: Collection-like abstraction for aggregates; hides persistence and querying details.

- Q: What is an anemic domain model?**
A: Data + getters/setters with logic in services. Leads to leaky models and duplication.
- Q: Domain event vs integration event?**
A: Domain event happens inside the domain. Integration event crosses context/service boundaries.
- Q: How do you maintain consistency across aggregates?**
A: Use transactions within aggregate; across aggregates use domain events, Sagas, Process Managers.
- Q: How do contexts relate on a context map?**
A: Partnership, Customer/Supplier, Conformist, ACL, or Shared Kernel.

8 PRODUCTION EXAMPLES

(SAME WORD "PRODUCT" MEANS DIFFERENT THINGS)

Context	What "Product" Means	Key Concerns	Example Aggregates
Catalog	A sellable item in the catalog.	Attributes, categories, media, status	Product (Catalog), Category, Brand, ProductImage
Pricing	Price list entry for a product or customer segment.	Price rules, currency, effective dates	PriceList, PriceRule, Discount, Money
Inventory	Stock-keeping unit (SKU) in a warehouse.	On-hand, reserved, locations, lots	InventoryItem, StockLevel, Reservation
Sales / Orders	Line items within a customer order.	Quantity, fulfillment, status	Order, OrderLine, Shipment, Return
Payments	The payable item or charge being paid.	Amount, method, reconciliation	Payment, Charge, Refund, Payout
Search Index	Document representing product for search.	Relevance, denormalized fields	ProductDocument, Facets, Synonyms
Analytics	Product as a dimension for reporting.	Metrics, aggregation, dimensions	ProductMetric, TimeSeries
Identity / Access	Entitlement or access to a product.	Permissions, plans, subscriptions	ProductEntitlement, Plan, AccessRight
Logistics	Physical good to be moved/delivered.	Dimensions, weight, handling	DimensionItem, Package, TrackingItem

9 INTERVIEW THINKING

- Identify domain & change pressure**
Where does the business change? What hurts?
- Learn the language with experts**
Elicit terms, meanings, and rules together.
- Define subdomains**
Core / Supporting / Generic.
- Carve bounded contexts & owners**
Set clear responsibilities and boundaries.
- Design aggregates & invariants**
What must always be true? Keep them small.
- Decide consistency model**
Transaction inside aggregate; eventual across.
- Define ports, events & policies**
Repositories, domain events, ACLs, policies.
- Map context relationships**
Partnerships, ACLs, shared kernels.
- Test the model & evolve**
Ubiquitous language, tests, and feedback loops.

7 RELATED

OOP	Clean / Hexagonal Architecture	CQRS	Event Sourcing
Microservices	DI	SOLID	Org Boundaries

10 SENIOR NOTES

- Bounded contexts are semantic boundaries, not deployment units.
- Aggregates protect invariants, not object graphs.
- Prefer small aggregates; start smaller than you think.
- Cross-context duplication is healthy and often necessary.
- Events require versioning, idempotency, and backward compatibility.
- DDD demands deep collaboration with domain experts. No experts, no DDD.

11 MENTAL HOOK

“DDD puts a boundary wherever the same word stops meaning the same thing.”

CLEAN CODE

SENIOR SOFTWARE ENGINEER / TECHNICAL LEAD INTERVIEW PREP (2-3 MINUTE REVIEW)

1 WHY

- Optimize code for safe reading, change, and operations.
- Reduce defect and coordination cost.
- Code is read far more than written.

READ : WRITE ESTIMATE

New code	~1 : 1
Maintenance	~10 : 1
Legacy systems	~20 : 1+

“ We do not pay for code when we write it, we pay for it every time we read or change it.”

4 COMPLEXITY

Same algorithmic Big O, different human cost. Optimize the whole lifecycle.

Human complexity	Cognitive load, branching, nesting
Change radius	Files, modules, and teams touched
Build / Test time	Feedback loop speed
Runtime abstraction cost	Indirection, allocations, coordination

Metrics (Signals, Not Targets)

Cyclomatic	McCabe complexity
Cognitive	Cognitive complexity (Sonar)
Coupling	Afferent (Ca), Efferent (Ce), Instability (I = Ce/(Ca+Ce))
Cohesion	LCOM, TCC
Churn	Change frequency, hotspots, ownership

Use metrics to start conversations, not to grade code.

7 RELATED

- Refactoring:** improve structure without changing behavior.
- SOLID:** Single Responsibility, Open/Closed, Liskov, Interface Segregation, Dependency Inversion.
- Design Patterns:** capture proven solutions; use sparingly.
- Testing:** unit, integration, contract, property, E2E.
- Static Analysis:** linters, type checkers, SAST, SCA.
- Observability:** logs, metrics, traces, alerts, dashboards.
- API Design:** clear contracts, versioning, backwards compatibility.
- Principles:** YAGNI (don't build unused), DRY (dup knowledge), KISS (keep it simple).

2 CORE IDEA

- Clear intent:** code communicates purpose.
- Small, coherent units:** one thing, well done.
- Explicit contracts:** types, pre/postconditions, and errors are clear.
- Consistent conventions:** predictable across the codebase.
- Local reasoning:** understand correctness with minimal context.

Clean code is contextual and evidenced, not aesthetic purity.

5 TRADE-OFFS

Optimize For	Watch Outs
Maintainability Reviewability Testability	Over-abstraction Premature generalization
Clear models Boundaries	Tiny-function fragmentation (call indirection tax)
Explicitness and clarity	Verbosity and noise
Performance and efficiency	Performance regression
Team alignment and consistency	Style debates that don't ship value

When NOT to "Clean"

- Generated / vendor / third-party code: wrap, don't churn.
- Hot paths: measure first, then make a targeted compromise.
- Prototypes / spikes: still set boundaries and name intent.

3 INTERNALS

- Naming & scope:** reveal intent; minimize mental mapping.
- Function cohesion:** do one thing; one level of abstraction.
- Control flow:** prefer straight-line; guard clauses; no hidden gotos.
- Data boundaries:** validate at edges; minimize leakage.
- Error semantics:** explicit, actionable, classifiable.
- Immutability:** prefer immutable data and functions.
- Dependency visibility:** inject; avoid hidden singletons.
- Comments explain why, not what.** Remove noise.
- Tests, static analysis, formatters:** automate consistency and correctness.
- Observability is production readability:** structured logs, metrics, traces, and clear signals.

6 DISCUSSION

- ? Is clean code subjective? How do we calibrate as a team?
- ? Comments: When helpful vs harmful? What to remove?
- ? Function length: What signals are we optimizing for?
- ? Naming: What makes a name good? Domain vs technical?
- ? DRY vs Duplication: Duplicate knowledge or code?
- ? Exceptions vs Result types: When and why?
- ? Testability: Design for test or test the design?
- ? Performance: How to balance clarity and speed?
- ? Legacy code: Strangler pattern? Boy Scout Rule?
- ? Code review standards: What do we block vs discuss?

8 PRODUCTION

- Service Handlers:** validate, authorize, idempotent, thin.
- Domain Rules:** pure, deterministic, well-named.
- Migration Scripts:** reversible, idle-safe, observable.
- SDK / Libraries:** stable APIs, minimal surprises.
- Infrastructure Code:** declarative, modular, reviewed.
- Runbooks / Code Links:** accurate, versioned, searchable.
- Logging & Errors:** actionable, structured, no PII.
- Database Queries:** parameterized, verified, explain plans.
- Concurrency Code:** races, deadlocks, cancellation, timeouts.

9 INTERVIEW THINKING

- Identify reader & change:** Who changes this? Why?
- Name intent:** Choose names that reveal purpose.
- Isolate invariant:** What must always be true?
- Simplify flow:** Remove nesting, clarify branches.
- Expose dependencies & errors:** Make them explicit.
- Add characterization tests:** Lock behavior before change.
- Measure:** Complexity, performance, and test impact.
- Small, reviewable diff:** Easy to read, easy to revert.
- Document rationale:** Why this approach, risks, trade-offs.

BEFORE (complex & implicit)

```
int calc(int a,int b){
  int r=0;
  if(a>0&&b>0){
    r=a*b;
  }else{
    for(int i=0;i<b;i++){
      r+=a;
    }
  }
  return r;
}
```

AFTER (clear & explicit)

```
int multiply(int a, int b) {
  if (a == 0 || b == 0) return 0;
  return a * b;
}
// Clear intent, guards, no nesting.
```

BEFORE (hard to change)

```
class OrderService {
  public void handle(Order o){
    validate(o);
    if(o.getType()=="R") {
      // 50 lines of rules
    } else {
      // more rules
    }
    repo.save(o);
    email.send(o); } }
```

AFTER (isolated & testable)

```
class OrderService {
  void handle(Order o){
    validate(o);
    rulesEngine.apply(o);
    repo.save(o);
    notifier.notify(o); } }
// Rules isolated and testable.
```

BEFORE (vague error)

```
if(user==null)
  throw new Exception(
    "Bad user");

if(amount<=0)
  throw new Exception(
    "Bad amount");
```

AFTER (actionable error)

```
if (user == null)
  throw new InvalidUserException(
    "userId is required");

if (amount <= 0)
  throw new InvalidAmountException(
    "amount must be > 0");
// Errors are part of your API.
```

10 SENIOR NOTES

- Optimize for the next change, not the shortest code.
- Duplication is cheaper than the wrong abstraction.
- Comments cannot rescue misleading code. Use them to explain constraints and decisions.
- Error messages are APIs—make them actionable.
- Consistency across the team beats personal taste.
- Clean code includes deletion. Stop carrying dead weight.
- Favor composability over inheritance.
- Make the easy path the right path.
- Cleanliness includes operability: deploy, observe, recover.
- Leave code better than you found it.

11 MENTAL HOOK

“ Clean code makes the next correct change obvious—and the dangerous change difficult.”

REFACTORING

Senior Software Engineer / Technical Lead Interview Preparation

1 WHY



Improve internal design without changing observable behavior.



Make future changes safer and faster.



Continuously pay down design friction.

2 CORE IDEA

Small behavior-preserving transformations with fast feedback.



Refactoring differs from:

- ✗ Feature = adds new user-visible behavior
- ✗ Rewrite = replaces system wholesale
- ✗ Optimization = changes performance characteristics

3 INTERNALS



IDE AST-aware transforms (rename, extract, inline, etc.).



Compiler / type checker enforces correctness.



Test safety net: unit, contract, property, snapshot tests.



Version control: small commits, clear messages, checkpoints.



Branch by abstraction: add indirection, route, then swap.



Strangler pattern for systems: slice & replace safely.



Database: expand → migrate → contract.



Observability (logs, metrics, traces) verifies behavior in production.

4 COMPLEXITY



Intended Big O unchanged unless explicitly an optimization.



Costs: engineering time, test runtime, change risk.



Benefits: lower change radius, lower cognitive load.

Behavioral equivalence includes:

- API surface and contracts
- Returned values and errors (types, messages)
- Timing/latency when contractual
- Side effects (I/O, DB, messages, files, logs)
- Concurrency behavior and ordering (when observable)

5 TRADE-OFFS



Trade-offs: maintainability/testability vs. regression risk, diff churn, merge conflicts, opportunity cost.



When NOT to refactor:

- No safety net (tests/observability) and high blast radius without first characterization.
- On-call / incident / deadline crunch.
- Generated code, vendor code, third-party libs.
- Code scheduled for deletion or replacement.
- When change scope is not well understood.

6 DISCUSSION



Refactor vs. rewrite: how do you decide?



How do you define and lock down behavior?



Legacy code with no tests: what's your plan?



How do you roll out a large refactor safely?



Database schema changes: expand-migrate-contract?



How do you use feature flags in refactoring?



How do you ensure performance doesn't regress?



How do you communicate and review refactors?



When do you stop refactoring?

7 RELATED



Clean Code (readability, naming, structure)



Testing (unit, integration, contract, property, snapshot)



SOLID principles (SRP, OCP, ISP, DIP)



Design Patterns (creational, structural, behavioral)



Technical Debt (interest vs. principal)



Strangler Pattern (incremental replacement)



Branch by Abstraction (seams, indirection)



Migrations (DB/schema, data, API versions)



Continuous Delivery (small, frequent, safe)

8 PRODUCTION REFACTORING EXAMPLES



Extract service/module from monolith



Replace conditionals with strategy/policy objects



API version migration (v1 → v2) with compatibility



Database schema evolution (expand → migrate → contract)



Dependency/library upgrades with adapters



Concurrency isolation (locks → queues/actors/transactions)



Logging & observability standardization



Modularize monolith (bounded contexts / modules)



Dead code removal and feature retirement

BEFORE / AFTER SNAPSHOTS

Extract Method

Before

```
function total(items) {
  let sum = 0;
  for (const i of items) {
    if (i.price > 0 && i.qty > 0) {
      sum += i.price * i.qty;
    }
  }
  return sum;
}
```

After

```
function total(items) {
  let sum = 0;
  for (const i of items) {
    if (isValid(i)) sum += lineTotal(i);
  }
  return sum;
}

function isValid(i) {
  return i.price > 0 && i.qty > 0;
}

function lineTotal(i) {
  return i.price * i.qty;
}
```

Replace Conditional with Polymorphism

Before

```
function discount(o) {
  if (o.type === 'VIP') return o.amount * 0.2;
  if (o.type === 'REGULAR') return o.amount * 0.1;
  if (o.type === 'NEW') return o.amount * 0.05;
  return 0;
}
```

After

```
class Customer {
  discount(amount) { return 0; }
}

class VipCustomer extends Customer {
  discount(a) { return a * 0.2; }
}

class RegularCustomer extends Customer {
  discount(a) { return a * 0.1; }
}

class NewCustomer extends Customer {
  discount(a) { return a * 0.05; }
}

function discount(a) { return o.discount(o.amount); }
```

Inline Temp → Improve Readability

Before

```
function obj(items) {
  const total = nums.reduce((a, n) => a + n, 0);
  const count = nums.length;
  return total / count;
}
```

After

```
function avg(nums) {
  return nums.reduce((a, n) => a + n, 0) /
    nums.length;
}
```

SAFE-CHANGE LOOP (DETAIL)



1 Characterize
• Read code
• Understand intent
• Capture behavior with tests & examples



2 Change One Thing
• Smallest possible transform
• Keep compile & tests green



3 Test / Measure
• Run full suite
• Add missing tests
• Check performance & static checks



4 Commit
• Small commit
• Clear message
• Easy to revert



5 Deploy & Observe
• Ship incrementally
• Monitor behavior in production
• Rollback plan

ROLLBACK PLAN



Feature flags



Blue/green or canary



Dark launch



Quick revert script

9 INTERVIEW THINKING

- 1 Define target: small, pain point, change pressure.
- 2 Define behavior contract (inputs, outputs, side effects).
- 3 Build / strengthen safety net (tests, characterization, observability).
- 4 Choose the smallest behavior-preserving transform.
- 5 Execute change; keep system always working.
- 6 Test, measure, and validate equivalence.
- 7 Commit with clear intent; document decisions.
- 8 Repeat the loop.
- 9 Deploy incrementally (feature flags, canary, blue/green).
- 10 Observe in production; have a rollback plan.

10 SENIOR NOTES

- Refactoring needs an economic reason (risk, cost, velocity).
- Separate behavior change from structure change when possible.
- Small, reversible steps beat heroic branches.
- Tests must cover observable contracts, not internals.
- Database & data migrations are refactors with temporal states — design for forwards & backwards compatibility.
- Delete before abstracting: remove dead code early.
- Communicate intent and trade-offs; socialize plan.

11 MENTAL HOOK

“Refactoring changes the path to future work without changing where today's code arrives.”